Project Technical Defense Guide: AMBA AXI4-Lite On-Chip BRAM Interface Subsystem IP

Dual-FSM Microarchitecture, 5-Channel Handshake Protocols, Memory Subsystem Interfacing, and Silicon Interview Defense

---

Anirudh Chaturvedi

M.Tech in Microelectronics & VLSI Design, Class of 2027
Indian Institute of Science (IISc), Bangalore

Digital Systems Design with FPGAs (E3 231) · DESE, IISc

Contents

1 AMBA AXI4-Lite Protocol Principles & System Specification **2**
1.1 Overview & System Motivation 2
1.2 Five Independent Channel Architecture 2
1.3 Handshake Invariants & Deadlock Prevention Rules 3

2 Dual-FSM Microarchitecture & Memory Subsystem **4**
2.1 Top-Level Hierarchy & Command Protocol 4
2.2 Master Finite State Machine ('AXI4$_L ITE_M ASTER$') 4
2.3 Slave Finite State Machine & BRAM Controller ('AXI4$_L ITE_S LAVE$') 5

3 FPGA Synthesis, Verification & Timing Analysis **6**
3.1 Simulation Stages on Artix-7 6
3.2 Resource Utilization & Timing Metrics 6

4 Silicon Interview Defense Dialogues **7**

Chapter 1

AMBA AXI4-Lite Protocol Principles & System Specification

1.1 Overview & System Motivation

In modern system-on-chip (SoC) architectures, high-performance interconnects require standardized, robust bus protocols to decouple compute masters from memory-mapped peripherals. The AMBA AXI4-Lite protocol provides a streamlined, point-to-point, register-slice-friendly memory-mapped interconnect subset tailored for control registers and localized on-chip memory blocks.

In this project, authored during the **Digital Systems Design with FPGAs (E3 231)** course at IISc Bangalore, a complete, synthesizable AMBA AXI4-Lite interface was architected from RTL primitives to manage bidirectional read and write access to a 32-bit $\times$ 256 on-chip Block RAM (BRAM) on the AMD Xilinx Artix-7 FPGA (‘XC7A35T‘).

1.2 Five Independent Channel Architecture

The AMBA AXI4-Lite specification defines five separate, unidirectional transaction channels. Every channel implements a two-way ‘VALID‘ / ‘READY‘ handshake mechanism:

1. **Write Address Channel (AW):** Carries the destination target write address (‘AWADDR[31:0]‘) and protection attributes (‘AWPROT[2:0]‘). Handshake: ‘AWVALID‘ (Master) and ‘AWREADY‘ (Slave).

2. **Write Data Channel (W):** Transfers the write data payload (‘WDATA[31:0]‘) and byte-enable write strobes (‘WSTRB[3:0]‘). Handshake: ‘WVALID‘ (Master) and ‘WREADY‘ (Slave).

3. **Write Response Channel (B):** Transmits the completion status of write transactions (‘BRESP[1:0]‘: ‘OKAY‘, ‘EXOKAY‘, ‘SLVERR‘, ‘DECERR‘). Handshake: ‘BVALID‘ (Slave) and ‘BREADY‘ (Master).

4. **Read Address Channel (AR):** Carries the source read memory address (‘ARADDR[31:0]‘) and protection signals (‘ARPROT[2:0]‘). Handshake: ‘ARVALID‘ (Master) and ‘ARREADY‘ (Slave).

5. **Read Data Channel (R):** Conveys the returned read data word (‘RDATA[31:0]‘) and transaction read status (‘RRESP[1:0]‘). Handshake: ‘RVALID‘ (Slave) and ‘RREADY‘ (Master).

1.3 Handshake Invariants & Deadlock Prevention Rules

The ARM AMBA specification mandates strict dependency rules to prevent circular combinational loops and system deadlock:

- **Invariant 1:** A source interface (‘VALID‘ sender) **must not** wait for the destination interface (‘READY‘ receiver) to assert before asserting ‘VALID‘. ‘VALID‘ can only be asserted based on internal state.
- **Invariant 2:** Once asserted, ‘VALID‘ must remain high until the transfer occurs (i.e., until the rising clock edge where both ‘VALID‘ and ‘READY‘ are logic ‘1‘).
- **Invariant 3:** A destination interface is permitted to assert ‘READY‘ before ‘VALID‘ is asserted, or wait until ‘VALID‘ is asserted before asserting ‘READY‘.

Chapter 2

Dual-FSM Microarchitecture & Memory Subsystem

2.1 Top-Level Hierarchy & Command Protocol

The top-level module (‘AXI4$_L ITE_T OP$‘)$coordinatescommunicationbetweentheverificationtestbenchhost, the$
$portBRAMcore.$

The verification host drives a 16-bit command bus (‘CMD$_I N$[15 : 0]‘) :

Bit [15] (R/W Flag): ‘0‘ indicates Write transaction; ‘1‘ indicates Read transaction.

Bits [14:8] (Word Count): 7-bit word transfer length (number of consecutive 32-bit words).

Bits [7:0] (Base Address): 8-bit memory starting word address (0 to 255).

2.2 Master Finite State Machine (‘AXI4$_L ITE_M ASTER$‘)

The Master FSM acts as the bus initiator. It transitions through the following discrete operational states:

- **M_IDLE:** Awaits host command assertion (‘START$_C MD = 1$‘).$Clearsinternaltransfercounters.$**M_**

- **M_WRITE_ADDR_DATA:** Asserts ‘AWVALID = 1‘ and ‘WVALID = 1‘ simultaneously. Coordinates address and data transmission with slave ‘AWREADY‘ and ‘WREADY‘.
- **M_WRITE_RESP:** Asserts ‘BREADY = 1‘. Waits for slave completion response (‘BVALID = 1‘ and ‘BRESP = 2’b00‘). Increments base address counter.
- **M_READ_ADDR:** Asserts ‘ARVALID = 1‘. Awaits slave address acknowledgement (‘ARREADY = 1‘).
- **M_READ_DATA:** Asserts ‘RREADY = 1‘. Awaits slave valid read data (‘RVALID = 1‘), captures ‘RDATA[31:0]‘ into host buffer, and checks ‘RRESP‘.
    - **M_BURST_NEXT:** Checks if all requested words have been transferred. Loops back to address phase if remaining words > 0; else returns to ‘M$_I DLE$‘$with$‘$CMD_D ONE$‘$pulsedhigh.$

2.3 Slave Finite State Machine & BRAM Controller (‘AXI4$_L ITE_S LAV$

The Slave FSM translates AXI4-Lite channel signals into native BRAM synchronous control strobes (‘BRAM$_E N$‘, ‘$BRAM_W E$‘, ‘$BRAM_A DDR$‘, ‘$BRAM_D IN$‘, ‘$BRAM_D OUT$‘) :

S_IDLE: Listens concurrently for write requests (‘AWVALID = 1‘) or read requests (‘ARVALID = 1‘).

S_WRITE_CAPTURE: Captures write address and write data. When both ‘AWVALID‘ and ‘WVALID‘ are affirmed, asserts ‘AWREADY‘ and ‘WREADY‘, activates ‘BRAM$_W E = 4'b1111$‘, $and writes ‘WDATA‘ into memory on the clock edge.$ **S_WRITE_RESP** $1‘ with ‘BRESP = 2'b00‘(‘OKAY‘). Holds until master responds with ‘BREADY = 1‘.$

S_READ_CAPTURE: Latches ‘ARADDR‘, asserts ‘ARREADY = 1‘, and drives synchronous BRAM read cycle (‘BRAM$_E N = 1$‘, ‘$BRAM_W E = 4'b0000$‘). **S_READ_DATA:** $Presents retrieved data on ‘RD$ $0]‘, asserts ‘RVALID = 1‘ and ‘RRESP = 2'b00‘, and holds until master samples with ‘RREADY =$ $1‘.$

Chapter 3

FPGA Synthesis, Verification & Timing Analysis

3.1 Simulation Stages on Artix-7

Functional verification was executed across three comprehensive stages in AMD Xilinx Vivado:

1. **Behavioral Simulation (RTL):** Verified clock-accurate protocol transactions, concurrent address/data handshakes, backpressure stall handling, and memory data integrity.
2. **Post-Synthesis Functional Simulation:** Validated gate-level synthesized netlist ensuring that state encoding and LUT inference preserved transaction invariants.
3. **Post-Implementation Timing Simulation:** Verified full routed netlist under real physical wire delays, setup/hold constraints, and clock skew targeting the Artix-7 FPGA (‘XC7A35T‘).

3.2 Resource Utilization & Timing Metrics

The complete subsystem demonstrates exceptional hardware efficiency, requiring minimal Slice LUTs and flip-flops, leaving the vast majority of FPGA fabric available for compute accelerators:

- **LUT Utilization:** $< 1.5\%$ of Slice LUTs (Master FSM, Slave FSM, address comparator, and channel multiplexers).
- **Memory Resources:** Exactly 1×36 Kb Block RAM configured as 32 bits $\times$ 256 deep single-port RAM.
- **Timing Slack:** Zero setup or hold timing violations on a standard 100 MHz clock constraint ($T_{clk} = 10$ ns), closing with abundant positive margin ($WNS > +5.0$ ns).

Chapter 4

Silicon Interview Defense Dialogues

Interview Defense 1: Channel Independence & Throughput

Interviewer: Why does AXI4-Lite define separate write address and write data channels instead of multiplexing them onto a single shared bus?
Candidate Defense: AXI4-Lite decouples address from data to maximize transfer flexibility and facilitate pipeline staging across complex SoC interconnects. In a high-speed interconnect fabric, the address decoder can evaluate routing paths and grant bus arbitrations before the master has finalized its data payload, or vice-versa. Furthermore, decoupling allows pipelined transactions: a master can issue write address $N + 1$ while data beat N is still transferring across the fabric, significantly improving bus throughput compared to rigid shared-bus architectures like APB.

Interview Defense 2: Protocol Deadlock Prevention Rules

Interviewer: Can an AXI Master assert ‘BREADY‘ only after ‘BVALID‘ goes high? Can ‘AWREADY‘ wait for ‘WVALID‘? How did you design your FSMs to avoid deadlock?
Candidate Defense: According to the AMBA AXI4 specification, a slave is allowed to wait for ‘VALID‘ before asserting ‘READY‘, but a master **must never** wait for ‘READY‘ before asserting ‘VALID‘. In our RTL implementation:

1. The master FSM asserts ‘AWVALID‘ and ‘WVALID‘ unconditionally upon entering the write phase, guaranteeing no dependency on slave readiness.

2. The master asserts ‘BREADY = 1‘ in anticipation of the response, preventing any slave stall.

3. On the slave side, the write channel evaluates both ‘AWVALID‘ and ‘WVALID‘. To prevent deadlocks, the slave asserts ‘AWREADY‘ and ‘WREADY‘ independently or simultaneously without requiring the master to assert one before the other.

Interview Defense 3: Comparison with Full AXI4 and APB

Interviewer: How does your AXI4-Lite implementation differ from Full AXI4 and ARM APB? When would you select AXI4-Lite over APB?
Candidate Defense:

- **Versus Full AXI4:** AXI4-Lite eliminates burst length capability (‘ARLEN‘/‘AWLEN‘ are fixed to 1 beat per handshake), removes transaction IDs (‘ARID‘/‘AWID‘, hence no out-of-order execution), and eliminates atomic locks (‘ARLOCK‘) and caching attributes. This strips away complex reorder buffers, reducing area by $> 80\%$.
- **Versus APB:** APB is a strict non-pipelined, two-phase protocol (SETUP then ACCESS) without backpressure on address phases. AXI4-Lite provides true bidirectional handshake capability on all channels, allowing peripherals to introduce arbitrary wait states via ‘READY‘ deassertion and integrate directly into high-speed crossbar matrices without protocol translation bridges.